AI-Powered UI Testing (Rupeek LP2) — Short Overview

What this project does

This repo runs **end-to-end UI tests** on Rupeek's LP2 booking flow (Fresh and Takeover).

Instead of hardcoding CSS selectors in code, we write a **JSON config** that describes:

- **Steps** to perform (click, fill, upload, navigate)
- **What should be true** after each step (assertions)
- **Test data** (mobile, PAN, pincode) using `.env` variables

Then the runner uses **Playwright** to drive the browser and **Claude (AI)** to "understand the page" and pick the right selector for each action.

Key idea: Config says *WHAT*, AI finds *HOW*

1) Config (WHAT to test)

Config files live in `config/`:

- `config/test-rules-fresh.config.json`
- `config/test-rules-takeover.config.json`
- `config/test-rules-state-management.config.json` (drop/resume, back, reload scenarios)

Each step looks like this (example shape):

- **Actions**: "click Get Started", "fill Mobile number input"
- **Assertions**: "element visible: Total Loan Amount", "url contains: /offer-page"

So the config is the **test script** and the **expected outcomes**.

2) AI planning (HOW to click/fill on the current page)

When the runner reaches an action like:

- `click` with label "Get Started"

it calls the AI planner (`planStep`) with:

- current page HTML
- current URL
- a screenshot
- the action label from config

Claude returns:

- a **CSS selector** to use
- which browser operation to run (click/fill/upload/check)
- optional value (for fill/select)

Then Playwright executes it.

Where AI (Claude) is used in our flow

In `src/agent.ts` we call Claude using the `ANTHROPIC_API_KEY`:

1. **`planStep`** — converts config labels → selectors (to execute actions)
2. **`evaluateAssertion`** — checks if an assertion passes using HTML + screenshot
3. **`diagnoseFailure`** — if a step fails, suggests recovery (retry/wait/skip/etc.)
4. **`generateReportSummary`** — writes a human-readable summary for the report

What Playwright does (browser automation)

Playwright is responsible for:

- Opening the browser
- Navigating pages
- Clicking/filling/uploading
- Taking screenshots
- Maintaining cookies/session in a browser context

Files:

- `src/browser.ts` (Playwright controller)
- `src/flow-step.ts` (runs actions + assertions for each step)
- `src/run-engine.ts` / `src/runner.ts` (runs the suite and generates report)

OTP handling

We support OTP in two ways:

- **CLI mode**: asks you in terminal (`INTERACTIVE_OTP=true`)
- **Chat Web UI**: asks you in a web popup (OTP modal)

OTP uses the config placeholder `{{prompt.otp}}`.

Reports

Every run produces:

- `report.html` — full interactive report with screenshots
- `result.json` — raw run results

Folder pattern:

- `reports/YYYY-MM-DD_HH-mm-ss_<variant>-<mode>/`

Why this approach is useful

- **Config-driven**: QA can change tests without editing TypeScript
- **Resilient**: AI can tolerate small UI changes (selectors change, labels remain)
- **Faster iteration**: failures show screenshots + step context
- **Covers complex flows**: drop/resume, reload, back navigation scenarios

---

One-line summary for demo

We write tests as a JSON checklist of user actions + expected results, and AI converts those labels into real browser selectors at runtime, so the automation stays stable even when the UI changes.